SdkLayout 2: Basic routing

Contents

This tutorial demonstrates how to define routes between the PEs of a code region using symbolic colors.

The key point here is that the colors that we use for the routes are symbolic (i.e., without a physical values). This means that the `SdkLayout` compiler will assign the value automatically.

For debugging purposes, the `SdkLayout` compiler will emit a JSON file called `colors.json` that contains the allocated physical color values.

send_receive.csl

```
// Select sender (0) or receiver (1)
param select: u16;
param c: u16;

const in_q = @get_input_queue(0);
const out_q = @get_output_queue(1);

const mode = enum(u16) {send = 0, receive = 1};

// Buffer to be sent
const size = 5;
const data = [size]u16{1, 2, 3, 4, 5};

// Buffer to receive data
export var buffer: [size]u16;

const dataDSD = @get_dsd(mem1d_dsd, .{.base_address = &data, .extent = size});
const bufferDSD = @get_dsd(mem1d_dsd, .{.base_address = &buffer, .extent = size});

const inDSD = @get_dsd(fabin_dsd, .{.extent = size, .fabric_color = @get_color(c), .input_queue
= in_q});
const outDSD = @get_dsd(fabout_dsd, .{.extent = size, .fabric_color = @get_color(c),
.output_queue = out_q});
```

```
// Sender task
const send_task_id = @get_local_task_id(8);
task send_task() void {
    @mov16(outDSD, dataDSD, .{.async = true});
}

// Receiver task
const receive_task_id = @get_local_task_id(9);
task receive_task() void {
    @mov16(bufferDSD, inDSD, .{.async = true});
}

const main_id = @get_local_task_id(10);
task main() void {
  // Select sender or receiver
  switch(@as(mode, select)) {
    mode.send => @activate(send_task_id),
    mode.receive => @activate(receive_task_id)
  }
}

comptime {
  @bind_local_task(send_task, send_task_id);
  @bind_local_task(receive_task, receive_task_id);
  @bind_local_task(main, main_id);
  @activate(main_id);

  @initialize_queue(in_q, .{.color = @get_color(c)});
  if (@is_arch("wse3")) {
    @initialize_queue(out_q, .{.color = @get_color(c)});
  }
}
```

run.py

```
#!/usr/bin/env cs_python

import argparse

import numpy as np

from cerebras.geometry.geometry import IntVector # pylint: disable=no-name-in-module
from cerebras.sdk.runtime.sdkruntimepybind import ( # pylint: disable=no-name-in-module
    Color,
    Route,
    RoutingPosition,
    SdkLayout,
    SdkTarget,
    SdkRuntime,
    SimfabConfig,
    get_platform,
)
```

```
parser = argparse.ArgumentParser()
parser.add_argument('--cmaddr', help='IP:port for CS system')
parser.add_argument(
    '--arch',
    choices=['wse2', 'wse3'],
    default='wse3',
    help='Target WSE architecture (default: wse3)'
)
parser.add_argument(
    '--cslc-prefix',
    type=str,
    default='',
    help='Optional path to bin/cslc-driver'
)
args = parser.parse_args()

############
### Layout
############
# If 'cmaddr' is empty then we create a default simulation layout.
# If 'cmaddr' is not empty then 'config' and 'target' are ignored.
config = SimfabConfig(dump_core=True)
target = SdkTarget.WSE3 if (args.arch == 'wse3') else SdkTarget.WSE2
platform = get_platform(args.cmaddr, config, target)
layout = SdkLayout(platform)

#################
### Code region
#################
# Create a 2x1 code region using 'send_receive.csl' as the source code.
# The code region is given the name 'send_receive'. The first PE will
# act as the sender and the second PE as the receiver.
code = layout.create_code_region('./send_receive.csl', 'send_receive', 2, 1)
# The 'set_param' method will set the value of a parameter on a specific
# PE using its local coordinates, i.e., the coordinates with respect
# to the respective code region and not the global coordinates.
# Here we set 'select=0' for the sender (coordinates (0, 0)) and
# 'select=1' for the receiver (coordinates (1, 0)).
# PE coordinates can be created using the 'IntVector' class that represents
# 2D grid coordinates.
sender_coords = IntVector(0, 0)
receiver_coords = IntVector(1, 0)
code.set_param(sender_coords, 'select', 0)
code.set_param(receiver_coords, 'select', 1)

########################################
### Routing between sender and receiver
########################################
# The sender routes traffic from the RAMP to the EAST where the
# receiver will receive the data (from the WEST) and route them to the RAMP.
send_routes = RoutingPosition().set_input([Route.RAMP]).set_output([Route.EAST])
receive_routes = RoutingPosition().set_input([Route.WEST]).set_output([Route.RAMP])
# Define a symbolic color. The SdkLayout compiler will resolve this into
# a physical value.
```

```
c = Color('c')
# Use color 'c' to paint the routes from sender to receiver.
code.paint(sender_coords, c, [send_routes])
code.paint(receiver_coords, c, [receive_routes])
code.set_param_all(c)

#################
### Compilation
#################
# Compile the layout and use 'out' as the prefix for all
# produced artifacts.
compile_artifacts = layout.compile(out_prefix='out', cslc_prefix=args.cslc_prefix)

############
### Runtime
############
# Create the runtime using the compilation artifacts and the execution platform.
runtime = SdkRuntime(compile_artifacts, platform, memcpy_required=False)
runtime.load()
runtime.run()
runtime.stop()

#################
### Verification
#################
# Finally, once execution has stopped, read the result from the receiver's
# memory and compare with expected value.
expected = np.array([1, 2, 3, 4, 5], dtype=np.uint16)
# The 'read_symbol' method will read a symbol from memory at the specified
# global coordinates and return it as a numpy array of type 'dtype'.
actual = runtime.read_symbol(1, 0, 'buffer', dtype='uint16')
assert np.array_equal(expected, actual)
print("SUCCESS!")
```

commands.sh

```
#!/usr/bin/env bash

set -e

cs_python run.py --arch=wse3
```


Last updated on Apr 15, 2026, 7:13:14 PM.